

Praktikum 10: Studi Kasus P2P Lending

Tugas Besar: Pengembangan Core Engine P2P Lending Platform

Mata Kuliah: Teknik Pemrograman

Program Studi: Sarjana Terapan Teknik Informatika

Durasi Pengerjaan: 6 Minggu

Jenis Pengerjaan: Tim/ Berkelompok (4-5 orang)

Narasi Kasus

Dunia finansial teknologi (FinTech) menuntut sistem yang tidak hanya cepat, tetapi juga memiliki integritas data yang absolut dan kode yang mudah dipelihara. Anda diminta untuk membangun Core Engine bagi sebuah platform Peer-to-Peer (P2P) Lending.

Platform ini mempertemukan Borrower (Peminjam) yang mengajukan pinjaman dengan Lender (Pendana) yang ingin menginvestasikan uangnya. Namun, tantangan utamanya terletak pada manajemen state yang kompleks dan aturan bisnis yang dinamis. Sebuah pinjaman memiliki siklus hidup yang ketat: mulai dari pengajuan yang harus divalidasi oleh Risk Engine, masa penawaran (Crowdfunding) di mana banyak lender bisa berkontribusi, proses pencairan dana (Disbursement), hingga skema pengembalian cicilan yang melibatkan perhitungan bunga dan denda keterlambatan.

Instruksi Pengerjaan (Persyaratan Teknis)

Dalam waktu 6 minggu, tim Anda yang terdiri dari 4-5 orang harus bisa menerapkan materi-materi pada mata kuliah ini, dengan memenuhi kriteria berikut: 1. **Desain**

Arsitektur & Pattern (The GoF Mandate).

- Sistem tidak boleh dibangun dengan pendekatan prosedural.
- Anda wajib mengintegrasikan minimal 5 GoF Design Patterns untuk menyelesaikan masalah spesifik: Gunakan State Pattern untuk mengelola transisi status Loan (misal: dari FUNDING tidak bisa langsung ke CLOSED tanpa melalui DISBURSED atau REPAYMENT). Gunakan Strategy Pattern untuk memisahkan algoritma perhitungan bunga (Fixed, Floating, atau Syariah). Gunakan Observer Pattern untuk sistem notifikasi saat sebuah funding terpenuhi.
- (Eksplorasi) Temukan tempat yang tepat untuk menerapkan Factory Method, Decorator, atau Chain of Responsibility.

2. Domain & Test Driven Development.

- Fokuslah pada Rich Domain Model. Pastikan logika bisnis seperti "validasi sisa limit pinjaman" berada di dalam Entity, bukan di Service yang anemis.
- Sebelum menulis kode, Anda mungkin perlu mendefinisikan Acceptance Criteria menggunakan format Gherkin (.feature files).

- Setiap fitur harus diawali dengan pembuatan Unit Test.
- Gunakan JUnit dan Mockito untuk melakukan mocking pada layer infrastruktur agar pengujian fokus pada logika domain.

3. Implementasi Kode & Transient (Volatile).

- Bahasa pemrograman wajib menggunakan Java.
- Seluruh data harus dikelola secara In-Memory menggunakan struktur data (seperti HashMap), namun tetap dibungkus dalam Repository Pattern agar layer aplikasi tidak mengetahui bahwa data tersebut disimpan di memori. Karena tidak ada DB, pengujian unit menjadi sangat krusial. Gunakan Mockito untuk menyimulasikan perilaku In-Memory Repository saat menguji layer Service.
- Terapkan Clean Code secara disiplin. Kode akan dinilai berdasarkan keterbacaan, penamaan yang deskriptif, dan ukuran method yang kecil. Lakukan Refactoring secara berkala
- Jika ditemukan Code Smells, gunakan teknik dari katalog Martin Fowler (misal: Extract Class, Replace Conditional with Polymorphism).

Deliverables (Output Tugas)

1. Source Code: Repositori Git dengan riwayat commit yang menunjukkan proses TDD dan Refactoring.
2. Dokumentasi Arsitektur: Penjelasan mengapa pattern tertentu dipilih dan bagaimana pemetaan Bounded Context dalam DDD.
3. Test Report: Laporan hasil pengujian JUnit dan Cucumber (Gherkin jika diperlukan).

Ruang Eksplorasi dan Kritisi

Terdapat beberapa ruang eksplorasi agar Anda bisa mengkritisi sistem yang hendak dibangun.

- Bagaimana sistem Anda menangani pembatalan pinjaman oleh peminjam saat dana sudah terkumpul 50%?
 - Bagaimana mekanisme pengembalian uang ke banyak lender secara otomatis?
- Anda bebas menentukan logika ini selama pola desain yang digunakan konsisten dan teruji.

Disclaimer

Bahwa instruksi tugas besar akan ada pembaruan. Serta, terbuka terhadap masukan dari mahasiswa.